

Milestone 4

Noah Stickel

University of Jamestown

Python

Jessica Schwartz

August 16, 2026

Bug 1:

Description: The confirmation message displayed after successfully adding an attendance record contained a spelling error: "Record successfull added."

Context: This bug appeared after entering a valid attendance record. The program reached the confirmation "print()" statement after creating and storing the record.

Symptoms: The program successfully added the record, but the confirmation message displayed to the user contained the misspelled word.

Root Cause and Fix: The root cause was a simple typing error in the output string. The word "succcessful" was changed to "successfully" so that the program displays a correctly spelled confirmation message.

Debugging technique: I tested the program using a valid attendance record and reviewed the output. By comparing the displayed message with the intended confirmation message, I identified the typo directly in the "print()" statement and corrected it.

Bug 2:

Description: The "count_attendance()" function did not correctly count all absences for an individual because the "return" statement was located inside the "for" loop.

Context: This problem appeared while testing the attendance-counting feature with multiple records for the same individual.

Symptoms: The function only checked the first record before returning the absence count. If the individual's additional records contained absences, those records were not checked.

Root Cause and Fix: The root cause was that "return absence_count" was indented inside the "for" loop. This caused the function to stop after processing the first record. The fix was to move the "return absence_count" statement outside the loop so every record was checked before the final count was returned.

Debugging Technique: I traced the function through multiple attendance records by hand and followed the loop one iteration at a time. This showed that the function immediately returned after the first record instead of continuing through the rest of the list.

Bug 3:

Description: The "search_attendance()" function printed "No records found" before it was finished checking all of the attendance records.

Context: The problem appeared when searching through multiple attendance records, particularly when the first record did not belong to the person being search for.

Symptoms: The program could display “No records found for ‘name’.” even though a matching record existed later in the attendance list.

Root Cause and Fix: The root cause was that “if not found:” was located inside the “for” loop. The program checked whether a match was found after each individual record instead of waiting until all records had been searched. The fix was to move the “if not found:” condition out the “for” loop.

Debugging Technique: I tested the search using a name that appeared in a later record rather than the first record. By tracing the value of “found”, I determined that it was still “False” after the first non-matching record, which caused the incorrect message to appear before the search was finished.

Bug 4:

Description: The “absence_alert()” function attempted to use “absence_count” and “name”, but those variables were not defined inside the function or passed to it as parameters.

Context: The problem appeared when the program attempted to execute the “absence_alert” function after searching for an individual.

Symptoms: The program produced a “NameError” because “absence_count” and “name” were not available inside the “absence_alert” function.

Root Cause and Fix: The root cause was that “absence_alert()” was defined as “def absence_alert(records):”, but the function used “absence_count” and “name” without receiving those values. The function should have received the individual’s name and calculated absence count as parameters, such as “absence_alert(name, absence_count)”, and then used those parameters when checking whether an alert was necessary.

Debugging Technique: I traced the variables from “searching_loop()” into “absence_alert()”. Although “absence_count” and “name” existed in “searching_loop()”, they were local variables and were not automatically available inside another function. This helped identify the need to pass the values as arguments.

Bug 5:

Description: The program called “count_attendance()” and “search_attendance()” at the bottom of the program without supplying the arguments required by their function definitions.

Context: The problem appeared in the program's main execution section after "main()" finished.

Symptoms: The functions required both "records" and "name", but the calls only provided "attendance". This caused the program to raise a "TypeError" because the required "name" argument was missing.

Root Cause and Fix: The root cause was a mismatch between the function definitions and the function calls. "count_attendance(records, name)" and "search_attendance(records, name)" required two arguments. The fix was to call these functions with the required arguments through "searching_loop()", where the user entered the name to search for, rather than calling them at the bottom without a name.

Debugging Technique: I compared each function definition with the corresponding function call and counted the required parameters. This allowed me to identify that the calls were missing the "name" argument and that the searching loop was the appropriate place to supply the user's search name.

Reflection and Patterns:

Pattern 1:

The first major pattern involved function control flow and indentation. The "return" statement in "count_attendance()" and the "if not found" condition in "search_attendance()" were placed inside loops when they needed to execute after the loops completed. These problems demonstrated how the location of a statement can significantly change how a function behaves.

Pattern 2:

The second major pattern involved passing information between functions. The absence-alert function needed the individual's name and absence count, but those values were not passed into it. The function calls at the bottom also did not provide all of the required arguments.

Lessons Learned:

The debugging process showed the importance of testing functions with multiple records instead of only one record. Testing only a single record could make the early "return" in "count_attendance()" appear to work correctly.

It also showed the importance of checking function definitions against function calls and making sure that values needed by one function are explicitly passed as parameters.

Future Actions:

1. Test each function with multiple inputs and edge cases, rather than only testing the simplest successful scenario.
2. Trace loops and conditional statements carefully to make sure “return” statements and final checks occur at the correct level of indentation.
3. Check function parameters and arguments whenever connecting multiple functions together to make sure the required data is being passed correctly.